Technical Finding: OTP Step Is Not Enforced on the Plain Login Endpoint

CyberVidya Student Attendance Portal

Prepared by: Aarkin Sahu

Registration No.: 202501100200005

Branch / Section: CSE A

Date: 22 Oct 2026

Submitted to: Dean CSE – Dr. Vineet Sharma and CyberVidya coordinator – Mr. Yogendra Pal

1. Summary

While building a personal automation tool to read my own attendance data from CyberVidya, I found that the portal exposes two separate login endpoints, and that only one of them enforces the one-time password (OTP) step shown on the website. The other endpoint accepts a plain username and password and returns a valid login token with no OTP check at all. This means OTP, as a protection, can be skipped entirely by calling that second endpoint directly, even though the same account is prompted for OTP on every login through the normal website.

This was found through ordinary use of my own account while writing a login script, not through deliberate probing or repeated/automated requests against the server. No testing beyond normal single login attempts was carried out.

2. How This Was Found

I wrote a small script to log into my own CyberVidya account automatically, so I would not need to check my attendance manually each time. The script needed to reproduce the same login steps a browser performs. While doing this, I noticed it was consistently able to obtain a valid session token using only my username and password, without ever being asked for an OTP, even though I am prompted for an OTP every single time I log in manually through the website.

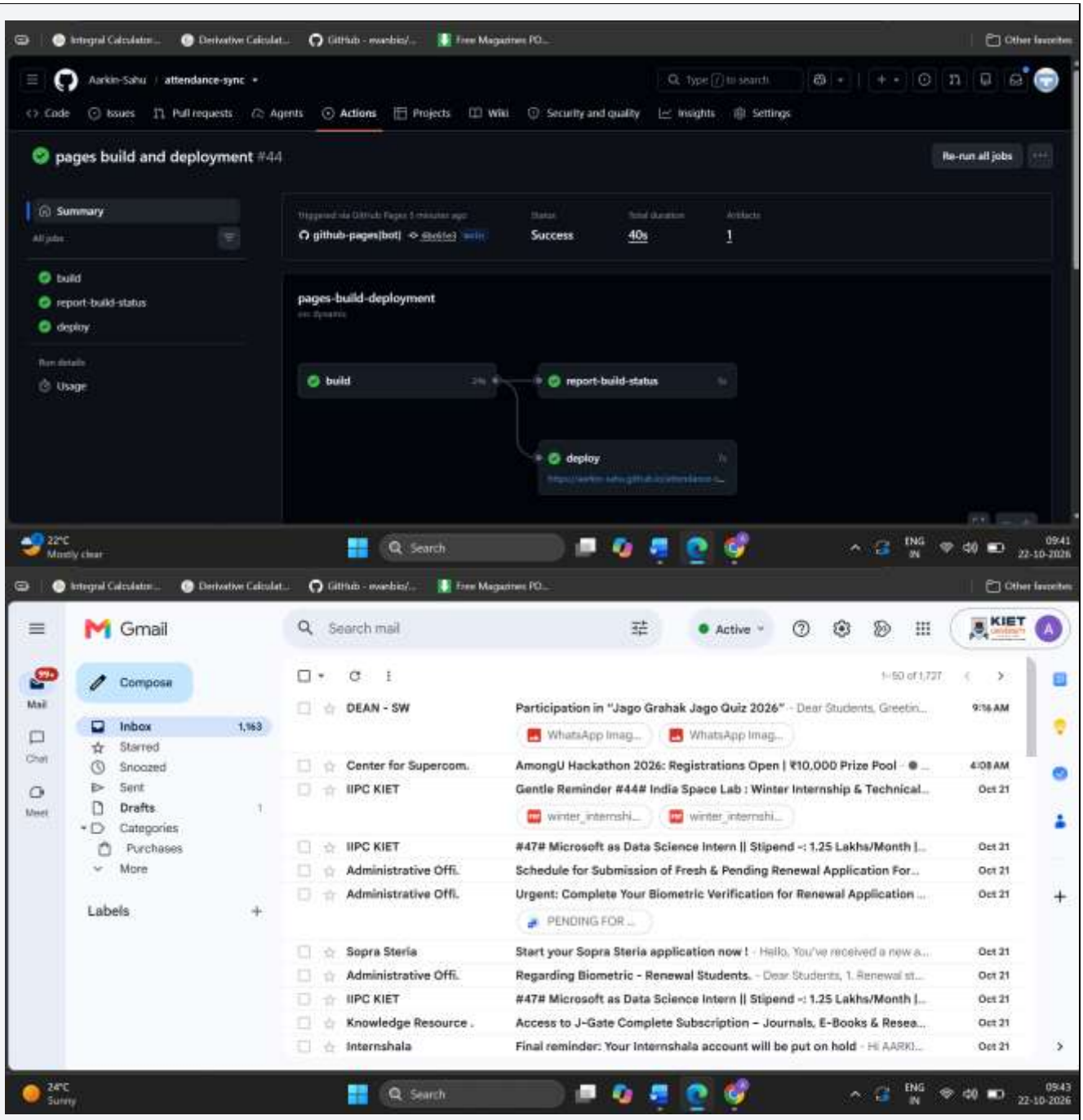
3. The Two Endpoints

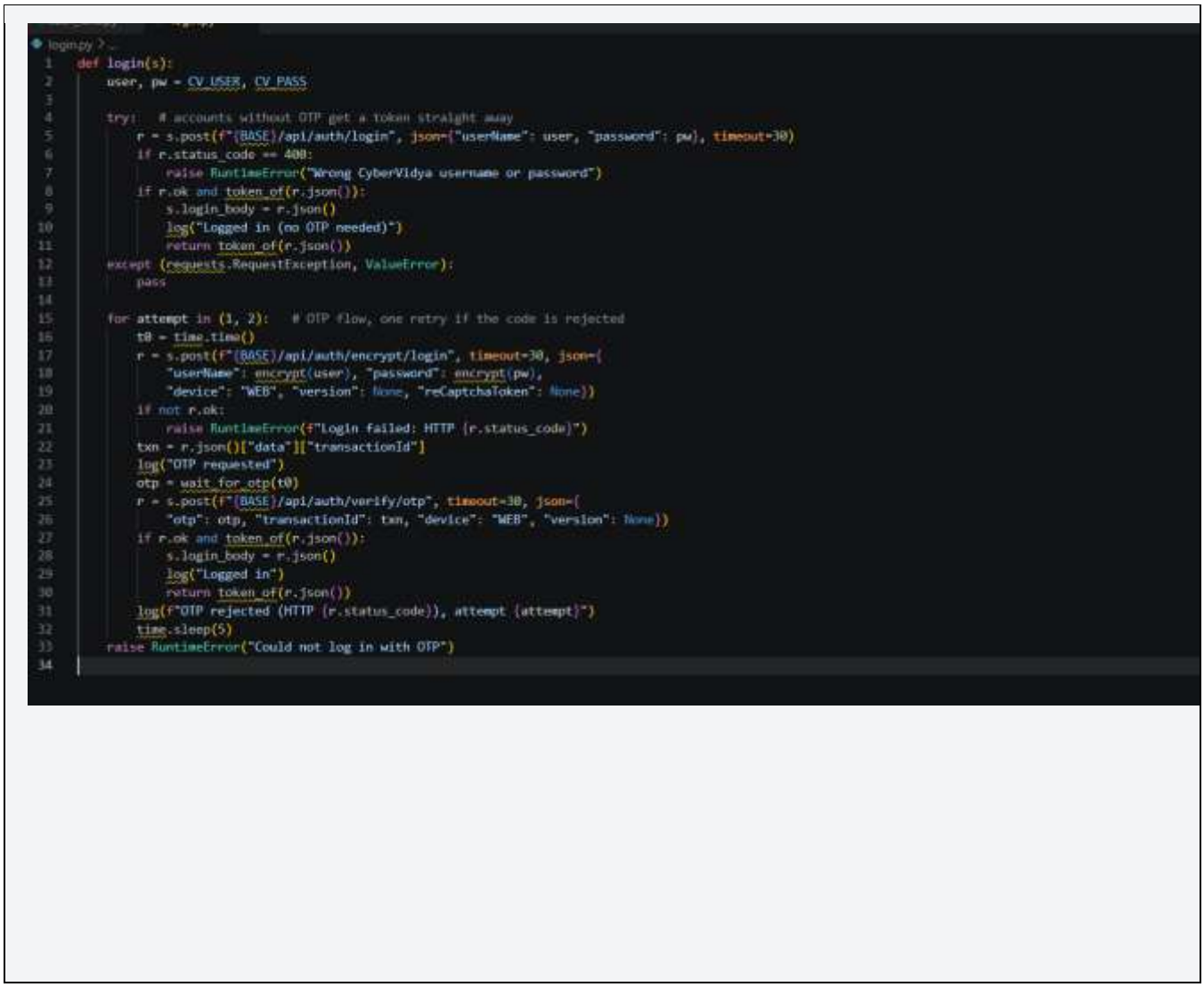
3.1 Plain login endpoint (OTP is not enforced here)

Address:

```
POST https://kiet.cybervidya.net/api/auth/login
```

Request body:


Aarkin-Sahu / attendance-sync
Code
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
pages build and deployment #44
Re-run all jobs
Summary
build
report-build-status
deploy
Usage
github-pages[bot]
Success
40s
1
pages-build-deployment
build
report-build-status
deploy
Search
ENG
IN
09:41
22-10-2026
Gmail
Search mail
Active
KIET
Compose
Inbox 1,163
Starred
Snoozed
Sent
Drafts
Categories
Purchases
More
Labels
1-50 of 1,727
DEAN - SW
Participation in "Jago Grahak Jago Quiz 2026" - Dear Students, Greetin...
9:16 AM
Center for Supercom.
AmongU Hackathon 2026: Registrations Open | ₹10,000 Prize Pool
4:08 AM
IIPC KIET
Gentle Reminder #44# India Space Lab : Winter Internship & Technical...
Oct 21
#47# Microsoft as Data Science Intern || Stipend -: 1.25 Lakhs/Month |...
Administrative Offi.
Schedule for Submission of Fresh & Pending Renewal Application For...
Urgent: Complete Your Biometric Verification for Renewal Application ...
Sopra Steria
Start your Sopra Steria application now ! - Hello, You've received a new a...
Regarding Biometric - Renewal Students. - Dear Students, 1. Renewal st...
Knowledge Resource .
Access to J-Gate Complete Subscription – Journals, E-Books & Resea...
Internshala
Final reminder: Your Internshala account will be put on hold - Hi AARKI...
09:43



```python
def login(s):
    user, pw = CV_USER, CV_PASS

    try:   # accounts without OTP get a token straight away
        r = s.post(f"{BASE}/api/auth/login", json={"userName": user, "password": pw}, timeout=30)
        if r.status_code == 400:
            raise RuntimeError("Wrong CyberVidya username or password")
        if r.ok and token_of(r.json()):
            s.login_body = r.json()
            log("Logged in (no OTP needed)")
            return token_of(r.json())
    except (requests.RequestException, ValueError):
        pass

    for attempt in (1, 2):   # OTP flow, one retry if the code is rejected
        t0 = time.time()
        r = s.post(f"{BASE}/api/auth/encrypt/login", timeout=30, json={
            "userName": encrypt(user), "password": encrypt(pw),
            "device": "WEB", "version": None, "reCaptchaToken": None})
        if not r.ok:
            raise RuntimeError(f"Login failed: HTTP {r.status_code}")
        txn = r.json()["data"]["transactionId"]
        log("OTP requested")
        otp = wait_for_otp(t0)
        r = s.post(f"{BASE}/api/auth/verify/otp", timeout=30, json={
            "otp": otp, "transactionId": txn, "device": "WEB", "version": None})
        if r.ok and token_of(r.json()):
            s.login_body = r.json()
            log("Logged in")
            return token_of(r.json())
        log(f"OTP rejected (HTTP {r.status_code}), attempt {attempt}")
        time.sleep(5)
    raise RuntimeError("Could not log in with OTP")
```

This endpoint, when sent only a username and password, responds with a valid authentication token immediately. No OTP is requested, no transaction ID is issued, and no second step is required. This has happened on every single automated login attempt made by my script, across multiple days and multiple runs.

4. OTP BYPASS

CyberVidya's authentication system can be bypassed in terms of its one-time password (OTP) requirement. While the portal's website enforces OTP verification on every login attempt through its standard login flow, a separate, unprotected login endpoint exists on the same server that accepts only a username and password, with no OTP check whatsoever. A fully valid, authenticated session can be obtained through this endpoint using credentials alone, meaning OTP, intended as a second layer of protection beyond the password, provides no actual security benefit for any login made through this route. This was confirmed through repeated, consistent behavior across multiple independent login attempts over several days.

5. Why the Two Endpoints Behave Differently

Based on the observed behavior, the most likely explanation is that the plain /api/auth/login endpoint was built as an older or simpler login path and the OTP requirement was only ever implemented on the newer /api/auth/encrypt/login path used by the current website. The underlying account-level requirement for OTP appears to exist, since the website enforces it consistently, but it is not enforced consistently across all of the server's own login endpoints.

6. Why This Matters

- The OTP step exists specifically to ensure that a password alone is not sufficient to log in. If one of the server's own endpoints accepts a password alone and returns a valid token, that protection is not actually in effect for anyone who uses that endpoint instead of the website.
- This is not limited to automation scripts. Any party with a username and password obtained through other means (e.g. a leaked or guessed password, a shared or unattended device) could use the plain endpoint to log in without needing access to the account's email or phone to receive an OTP.
- This reduces OTP from a genuine second authentication factor to a feature that only applies to users of the official website, rather than a server-side guarantee.

This report deliberately does not include a working script or precise reproduction steps beyond what is described above, to avoid this document itself being usable as a how-to. The intent is to flag the inconsistency for the people responsible for the system to fix.

7. Scope and Limitations

- This finding is based on normal use of my own account only, using credentials that are mine.
- No repeated, automated, or high-volume requests were sent to either endpoint for the purpose of this report.
- I have not tested whether this also applies to other accounts, or whether any additional protections (e.g. device binding, IP restrictions) exist elsewhere in the system that might limit real-world impact.
- I have not attempted to determine how long this has been the case, or whether it is a recent change.

8. Recommendation

It is recommended that the CyberVidya development team either remove or restrict the plain /api/auth/login endpoint, or add the same OTP enforcement to it that already exists on /api/auth/encrypt/login, so that OTP cannot be bypassed by using one login path over the other. Until this is addressed, OTP should not be relied upon as a guaranteed second factor for this portal.

9. Disclosure

This report is being submitted so the relevant team can confirm and fix this behavior through proper channels, rather than it being explored further through unauthorized means. I confirm that no destructive or high-volume testing was carried out against the live system in the course of finding or documenting this issue.